

计算机设计与实践

单周期CPU设计(EX、MEM、WB)

2026 · 夏

哈工大



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HIT

实验目的

- ◆ 通过模块化设计方式，加深对CPU结构和工作原理的理解
- ◆ 掌握根据数据通路表和控制信号取值表来实现*执行*、*访存*和*写回*

单元的方法

- ◆ 熟练掌握使用Verilog HDL实现CPU的功能部件



实验内容

- ◆ 使用Verilog HDL实现单周期CPU的执行单元:
 - ◆ 根据数据通路表和控制信号取值表，确定执行单元的接口
 - ◆ 实现所需的功能部件（如ALU, etc.）

执行单元	
ALU	
A	B

- ◆ 根据数据通路表，连接各个部件，形成执行单元

实验内容

- ◆ 使用Verilog HDL实现单周期CPU的访存单元：
 - ◆ 根据数据通路表和控制信号取值表，确定访存单元的接口
 - ◆ 实现所需的功能部件（如MREQ、MEXT, etc.）

存储单元	
MREQ	MEXT

- ◆ 根据数据通路表，连接各个部件，形成访存单元
- ◆ 使用Verilog HDL实现单周期CPU的写回逻辑



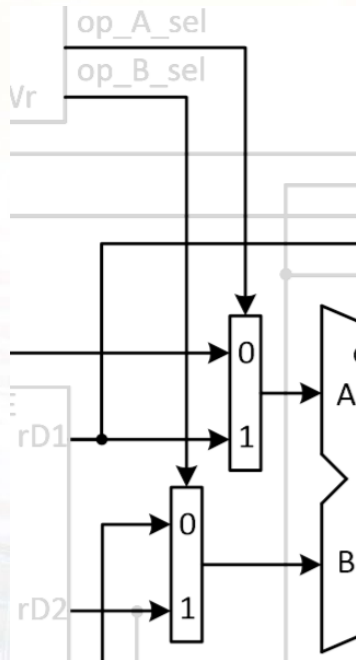
执行单元实现 – 关键点

◆ 选择运算数据

- ◆ **要点：** 根据控制器的操作数选择信号，选择不同的源操作数：

```
input [31:0] rd_dat1_i; // (rs1)
input [31:0] rd_dat2_i; // (rs2)
input [31:0] npc_i;     // Next PC
input [31:0] imm_i;     // 扩展后的立即数
```

```
assign op_A = op_A_sel ? npc_i : rd_dat1_i;
assign op_B = op_B_sel ? imm_i : rd_dat2_i;
```



执行单元实现 – 关键点

- ◆ 选择运算数据
- ◆ 完成算术、逻辑、移位运算
 - ◆ **要点：**调用设计好的ALU模块；用控制器的op信号选择运算功能
- ◆ 完成比较运算的PC值计算（也可在NPC实现）
 - ◆ **要点：**B型、J型指令的立即数在符号扩展后，需要加上PC的值
- ◆ 运算结果输出（含比较后赋值）
 - ◆ **要点：**根据控制器的ALUCtrl等控制信号，输出相应的运算结果
- ◆ 基本思路：先实现所需部件，再根据数据通路表连接起来



构建功能部件 – ALU

- ◆ ALU的功能:

- ◆ 算术运算 (add、sub, etc.)
- ◆ 逻辑运算 (and、or, etc)
- ◆ 移位运算 (sll、srl, etc)
 - ◆ 算术右移: $\$signed(a) >>> b$
- ◆ 比较运行 (slt、sltu, etc)
 - ◆ 有符号比较: $\$signed(a) > \$signed(b)$
 - ◆ 无符号比较: $\$unsigned(a) > \$unsigned(b)$



构建功能部件 – ALU

- ◆ 乘除法指令：要求自行设计乘除法器实现，不得用运算符、IP核
 - ◆ 除法器可基于恢复余数法、加减交替法实现
 - ◆ 原码除法回顾
 - 符号与数值分开运算：
 - 商的符号 = 被除数的符号 **异或** 除数的符号
 - 商的值 = 被除数的绝对值 除以 除数的绝对值
 - 余数的符号与被除数相同

构建功能部件 – ALU 原码除法

□ 恢复余数法

- ① 被除数加上 $[-y^*]_{\text{补}}$
- ② 若被除数(余数)为负, 商上0,
加上 $[y^*]_{\text{补}}$ 以恢复余数;
若被除数(余数)为正, 商上1;
- ③ 若左移次数小于 n , 不含符号的位宽
令被除数(余数)左移1位;
否则结束

【例】 $x = 13, y = -6$, 求 $z = [x \div y]_{\text{原}}$

$[x]_{\text{原}} = 01101,$

$[y^*] = 00110, [-y^*]_{\text{补}} = 11010$

$z_0 = x_0 \oplus y_0 = 0 \oplus 1 = 1$, 故商为负

被除数 (余数)	商	说明
$\begin{array}{r} 00001101 \\ +11010 \\ \hline \end{array}$		$+[-y^*]_{\text{补}}$
$\begin{array}{r} 110101101 \\ +00110 \\ \hline \end{array}$	___0	余数为负, 不够减, 上商0 恢复余数: $+ [y^*]$
$\begin{array}{r} 000001101 \\ 000011010 \\ +11010 \\ \hline \end{array}$	__0_	恢复余数后, 商和余数左移1位 $+ [-y^*]_{\text{补}}$
$\begin{array}{r} 110111010 \\ +00110 \\ \hline \end{array}$	__00	余数为负, 不够减, 上商0 恢复余数: $+ [y^*]$
$\begin{array}{r} 000011010 \\ 000110100 \\ +11010 \\ \hline \end{array}$	_00_	恢复余数后, 商和余数左移1位 $+ [-y^*]_{\text{补}}$
$\begin{array}{r} 111010100 \\ +00110 \\ \hline \end{array}$	_000	余数为负, 不够减, 上商0 恢复余数: $+ [y^*]$
$\begin{array}{r} 000110100 \\ 001101000 \\ +11010 \\ \hline \end{array}$	000_	恢复余数后, 商和余数左移1位 $+ [-y^*]_{\text{补}}$
$\begin{array}{r} 000001000 \\ 000010000 \\ +11010 \\ \hline \end{array}$	0001 001_	余数为正, 上商1 商和余数左移1位 $+ [-y^*]_{\text{补}}$
$\begin{array}{r} 110110000 \\ +00110 \\ \hline \end{array}$	0010	余数为负, 不够减, 上商0 恢复余数: $+ [y^*]$
$\begin{array}{r} 000010000 \end{array}$		

构建功能部件 – ALU 原码除法

□ 加减交替法

- ① 被除数加上 $[-y^*]_{\text{补}}$
- ② 若被除数 (余数) 为负, 商上0,
被除数 (余数) 左移1位后 $+[y^*]_{\text{补}}$;
若被除数 (余数) 为正, 商上1,
被除数 (余数) 左移1位后 $[-y^*]_{\text{补}}$;
- ③ 若左移次数小于n, 回到②; 否则结束

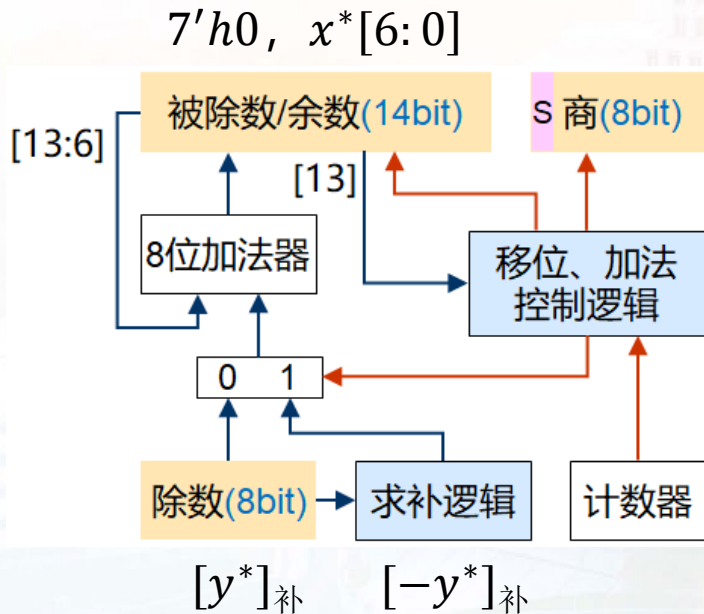
【例】 $x = 13$, $y = -6$, 求 $z = [x \div y]_{\text{原}}$
 $[x]_{\text{原}} = 01101$,
 $[y^*] = 00110$, $[-y^*]_{\text{补}} = 11010$
 $z_0 = x_0 \oplus y_0 = 0 \oplus 1 = 1$, 故商为负

被除数 (余数)	商	说明
$\begin{array}{r} 00001101 \\ +11010 \end{array}$		$+[y^*]_{\text{补}}$
$\begin{array}{r} 110101101 \\ 101011010 \\ +00110 \end{array}$	$\begin{array}{r} _ _ _ 0 \\ _ _ 0 _ \end{array}$	余数为负, 不够减, 上商0 商和余数左移1位 $+[y^*]$
$\begin{array}{r} 110111010 \\ 101110100 \\ +00110 \end{array}$	$\begin{array}{r} _ _ 00 \\ _ 00 _ \end{array}$	余数为负, 不够减, 上商0 商和余数左移1位 $+[y^*]$
$\begin{array}{r} 111010100 \\ 110101000 \\ +00110 \end{array}$	$\begin{array}{r} _ 000 \\ 000 _ \end{array}$	余数为负, 不够减, 上商0 商和余数左移1位 $+[y^*]$
$\begin{array}{r} 000001000 \\ 000010000 \\ +11010 \end{array}$	$\begin{array}{r} 0001 \\ 001 _ \end{array}$	余数为正, 上商1 商和余数左移1位 $+[y^*]_{\text{补}}$
$\begin{array}{r} 110110000 \\ +00110 \end{array}$	$\begin{array}{r} 0010 \end{array}$	余数为负, 不够减, 上商0 $+[y^*]$
$\begin{array}{r} 000010000 \end{array}$		

构建功能部件 – ALU 原码除法

❑ 除法器参考架构

- 原码除法的基本操作：求补、加法、左移、异或



① 复位时

- 计数器初始为6，其余清零

② 运算时

- 商符S通过异或求得
- 控制逻辑根据被除数 (余数) 的符号，选择加法的数据源，并控制上商
- 控制逻辑根据计数器值，控制被除数 (余数) 的移位次数

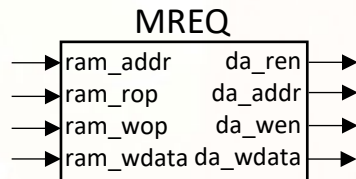
构建功能部件 – ALU 原码除法

□ 除法器接口与乘法器相同

序号	名称	位宽	属性	功能描述
1	clk	1	输入	时钟信号
2	rst	1	输入	复位信号（高电平有效）
3	x	8	输入	被除数（原码）
4	y	8	输入	除数（原码）
5	start	1	输入	启动信号
6	z	8	输出	商（原码）
7	r	8	输出	余数（原码）
8	busy	1	输出	忙标志信号

构建功能部件 – MREQ

- ◆ MREQ功能：根据访存指令需求，发出访存请求
 - ◆ 访存请求的关键：da_ren、da_wen，分别用于读请求、写请求
 - ◆ 发出的访存地址da_addr应是字地址
 - ◆ 读访存：ram_rop有效时，置da_ren为4'hF
 - ◆ 写访存：根据ram_wop和访存地址偏移量，生成da_wen：



指令	访存地址[1:0]	da_wen	完成的写操作
sb	2'b00	4'b0001	wdata[7:0]写入对应存储单元的[7:0]
	2'b01	4'b0010	wdata[7:0]写入对应存储单元的[15:8]
	2'b10	4'b0100	wdata[7:0]写入对应存储单元的[23:16]
	2'b11	4'b1000	wdata[7:0]写入对应存储单元的[31:24]
sh	2'b00	4'b0011	wdata[15:0]写入对应存储单元的[15:0]
	2'b10	4'b1100	wdata[15:0]写入对应存储单元的[31:16]
	其他	4'h0	地址不对齐，不访存
sw	2'b00	4'b1111	wdata写入对应存储单元的[31:0]
	其他	4'h0	地址不对齐，不访存

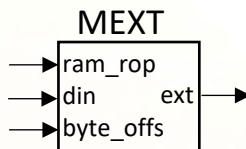
构建功能部件 – MEXT

- ◆ MEXT功能：对读访存的数据进行符号扩展

- ◆ 读访存请求发出的访存地址是字地址

- ◆ LW指令取回的是32位数据，不需扩展

- ◆ 对于LH、LHU、LB、LBU，需根据地址偏移量扩展：



指令	访存地址[1:0]	扩展操作
lb、lbu	2'b00	对读回数据的[7:0]进行符号扩展或零扩展
	2'b01	对读回数据的[15:8]进行符号扩展或零扩展
	2'b10	对读回数据的[23:16]进行符号扩展或零扩展
	2'b11	对读回数据的[31:24]进行符号扩展或零扩展
lh、lhu	2'b00	对读回数据的[15:0]进行符号扩展或零扩展
	2'b10	对读回数据的[31:16]进行符号扩展或零扩展
	其他	非法输入，可输出某个特定的常数值，方便调试

实验步骤

- ① 根据数据通路表，确定执行、访存单元**包含哪些部件**
 - ② 根据数据通路表，确定各功能部件的**接口**和需要实现的**功能**
 - ③ 在模板工程上，使用Verilog HDL实现各功能部件
 - ④ 根据数据通路表，将各功能**部件连接**起来
 - ⑤ **封装**成模块，得到执行单元和访存单元
 - ⑥ 根据数据通路表，连接相应的信号，实现写回逻辑
- ◆ **注意代码规范性，可以避免很多问题！**



课堂作业

- ◆ 将自己设计的ALU和执行单元画出来，包括：
 - ◆ 执行单元与其他单元 (取指、译码, etc.) 的交互信号
 - ◆ ALU内部子模块及模块之间的连接

所画的图要和数据通路表对应！





HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ